

Problem Solver Platform: Evaluation of the General-Purpose Solver System

Date: 4 October 2026 · **System:** Problem Solver platform at `D:\solver` (build of 4 Oct 2026, migration head 0107) · **Language model:** the platform's Assistant on vLLM `qwen3.5` (32k context) · **Method:** live field tests on the running platform, each compared with a solution computed independently (Python + HiGHS), plus the automated test suites.

1. Summary

The platform's goal is to take a decision problem described in plain words, with the user's data files, and turn it into a correct optimisation model. It should then solve that model and explain the answer. The problems are a mix of the general-purpose solver (the modelling language, compiler and solver back ends) and the Assistant that writes models in that language.

Verdict. On the final build, the system solved both test problems exactly as the independent reference did: the same goal values and the same decisions, end to end from a plain-language description. The **solving core is sound**: whenever the model was right, the solver's answer was right, in under a second. The weak point is the step from words to model. A 32k-context local model writes the model language slowly and with slips. The platform now catches, repairs or exposes most of those slips: 30 defects found in the field tests were fixed, and a read-back of every plan was added. Some modelling errors remain that no automatic check can see; a person reading the plan's read-back before approving it is still part of a correct result.

Area	Rating (1-5)	In one line
Solver core (compiler, back ends, results)	5	Every correctly modelled problem was solved exactly and fast.
Modelling language (IR)	4	Expressive enough for both test classes; a few natural forms are now accepted as well.
Data intake (files, maps)	4	CSV/Excel/GIS read well; exact totals and queries now available to the model.
Assistant: interview	4	Asks the right questions, finds infeasibility early; over-asking fixed.
Assistant: writing the model	3	Gets there, but needs 2-6 checking rounds and makes semantic slips.
Verification and safeguards	4	Dry runs, data checks, read-back, approval gate; semantic errors still need a reader.
Reporting the answer	4	Joined, exact result tables (<code>read_result</code>); earlier reports had factual slips.
Speed (with Qwen 3.5)	2	6 min (warehouses) to 12 min (roster) per problem; each model reply takes 1-5 min.
Robustness	4	3 crash paths fixed (save overflow, context overflow, empty-reply exhaustion).
Engineering quality	5	4,779 backend and 3,379 frontend tests pass; changes have regression tests.

Area	Rating (1-5)	In one line
Overall	4 / 5	Correct and trustworthy with a person approving the plan; slow with the current model.

2. What was evaluated

2.1 The system

Part	Size	Role
Backend (FastAPI, Python)	225 modules, ~54,700 lines; 265 API routes	Domains, records, parameters, problems, models, scenarios, runs, GIS, the Assistant
Frontend (React, TypeScript)	295 modules, ~68,400 lines	Model Editor, Map data, runs, the Assistant panel
Database	PostgreSQL, 107 migrations; ClickHouse for analytics	Data, models, frozen datasets, results
Modelling language (Problem IR v2)	10 term kinds (const, par, var, attr, sum, add, mul, pwl, fn, predict); binary / integer / continuous / interval decisions; weighted and ranked (lex) goals; 142 refusal rules	What every model is written in
Solver back ends	CP-SAT, HiGHS, GLOP, PDL, SCIP, Ipopt, MILP, Benders, and heuristics (GA, PSO, CMA-ES)	Chosen per model by the selector
Assistant	app/agent/ with tools: describe_workspace, read_file, query_file , check_spec, propose_plan, read_result , call_api, place_file	Interview → model → approval → build → solve → report

2.2 The test problems

	Warehouse location	Nurse roster
Class	Mixed-integer facility location and assignment	Time-indexed scheduling, soft requests, ranked goals
Data (attached CSV)	5 warehouses, 12 customers, 60 lanes	9 nurses, 21 day×shift cover rows, 8 day-off requests
Told only when asked	max 3 open; no lane > 450 km; one warehouse per customer	one shift a day; no morning after a night; night pays 1.3×; requests soft; goals in order
What makes it hard	a rule on a value per <i>pair</i> (km per lane)	consecutive days, a rate by kind of shift, a list as a 0/1 table, two ranked goals
Independent reference	Open Cairo-60Oct, Tanta, Assiut; 1,338,765 EGP/month (fixed 860,000 + transport 478,765)	2 requests broken (Friday seniors), then 37,650 EGP

The reference answers were computed outside the platform (PuLP + HiGHS) and never shown to the Assistant.

2.3 Method

Each run was driven like a real user would drive it: a vague first message with the files attached, then the facts it asked for, then approval of its plan. The answers were compared with the reference. After

each run the defects were fixed in the platform, covered by tests, deployed, and the problem was run again from scratch. Both problems were also written by hand in the modelling language and solved on the platform, which checks the solver core separately from the Assistant.

3. Results

3.1 Final retest (current build)

	Warehouse	Roster
Interview	1 round, correct totals (2,850 t capacity vs 1,655 t demand)	1 round; found Friday's senior shortage itself
Checking rounds before a valid model	2	6
Model correct?	Yes	Yes (confirmed through the read-back)
Result	1,338,765 EGP, same 3 warehouses, same 12 assignments	2 broken (Amal and Eman, Friday), 37,650 EGP
Matches reference	Exactly	Exactly
Time, first message to answer	~6 min	~12 min

3.2 How the runs went

Warehouses

Run	Outcome
1 (before)	Arithmetic wrong twice; then 28,000 characters of "thinking aloud" shown as the answer, twice; never produced a model.
2	Correct numbers; model passed every check but was wrong (a cost parameter never loaded: every lane cost 999,999,999); every solve crashed while saving the result.
3	Correct model; 1,338,765, exactly the reference. ~15 min.
Retest	Correct; exact; ~6 min.

Roster

Run	Outcome
1	Good interview; three long specs refused for JSON slips; turn ended in empty replies.
2	Correct numbers; model wrong but accepted (goal counted every shift, not broken requests: "47"); week wrapped Saturday→Sunday; turn crashed on context overflow.
3	Correct; exact (goal split in two, harmless here). ~25 min.
Retest A	Plan wrong but accepted : one 1.3× multiplier for every shift; rest rule reversed. Caught by comparison → read-back added. Not approved.
Retest B	Correct, verified through the read-back; exact; ~12 min.

3.3 The solver core alone

Both problems, written by hand in the modelling language with the patterns the Assistant is now taught, build and solve on the platform to the reference values. These are automated tests:

`test_the_field_test_problem_builds_and_solves_to_the_reference` and

`test_the_roster_built_with_the_patterns_solves_to_the_reference`. Solve times were 0.01–0.1 s (CP-SAT, HiGHS). The solver core produced **no wrong answer** in any run. Every wrong result traced back to a wrong model.

4. Evaluation by area

4.1 Solver core: 5/5

- Correct on every correct model. The selector picks a suitable back end (CP-SAT for the binary models; HiGHS for the ranked roster).
- Results carry status, optimality, bound, rule slack, the goal broken down by part, and conflicts for infeasible runs.
- **Defect found:** quantities were `numeric(15,6)`, which holds 9 digits before the point, not 15. Any objective or value of 1 billion or more crashed the run *after* it solved. Fixed in migration 0107 (`numeric(24,6)`). The worker also now records the real error instead of "transaction aborted".

4.2 Modelling language: 4/5

- It expressed both classes cleanly. Rules across consecutive periods use relationships (`next_day + via`); one particular record is picked with `where id = "night"`; ranked goals use `lex`; three-factor products nest.
- No conditional (`if/else`) exists; Qwen reached for one repeatedly. A parameter indexed by the kind of record is the right way, and the refusal message now says so.
- Natural forms added: `x[n, d, "night"]` with a key where an index belongs is bound automatically, and `"value": 1` loads a file that only lists pairs.
- Strict validation (142 rules) is a strength. Several messages were too terse for a model to act on, and are now written so they say how to fix the problem.

4.3 Data intake: 4/5

- CSV, Excel, JSON and map files (DXF, GeoJSON, KML, Shapefile, GeoPackage) are read into tables, and rows load straight from files into the model, so data never passes through the chat.
- **Added:** exact totals per column for every attached sheet, and `query_file` (filter, count, sum, group). These removed the arithmetic errors completely in later runs.
- **Fixed:** a key repeated in a sheet (days in a day×shift table) becomes one record.
- Noise: a CSV with latitude/longitude columns is treated as a map file, which adds extra columns.

4.4 Assistant: interview: 4/5

- It keeps an "Agreed so far" list, asks at most 4 numbered questions, and sizes the problem early: it found the warehouse capacity margin and Friday's senior shortage without being told.
- It over-asked in places: a coordinate system for a roster, a domain name, questions already answered. A prompt rule now stops this, and the retests asked one round each.

4.5 Assistant: writing the model: 3/5

- **Slow and slip-prone.** A roster spec is ~10,000 characters, and each reply takes Qwen 1–5 minutes. Typical slips were closers in the wrong order, a dropped `"right":` key, invented syntax,

keys used as indices, and lists split over duplicate keys. These are now repaired (spec tools only) or explained with the fix, so most cost one round instead of three.

- **Semantic slips the checks cannot see:** goals measuring the wrong thing, a rule in the wrong direction, a rate applied to every record. Three of four such slips were caught only by comparing with the reference. Two are now caught automatically (unloaded parameter; loaded data never read). The rest are exposed by the read-back (§4.6).

4.6 Verification and safeguards: 4/5

- In order: an interview before any plan; a dry run of every spec; data checks; a read-back of the spec; the person's approval; an all-or-nothing build; the user's own permissions and audit trail.
- **Read-back (new):** the platform renders the spec as formulas, for example "night_multiplier: ONE number for the whole problem = 1.3" or " $d2 \in \text{day that } d1 \text{ links to by next_day } (d1 \rightarrow d2)$ ". It goes back to the model with an instruction to compare, and into the plan the person approves under **As built**. In Retest B the model's summary and the read-back agreed, and both were correct.
- Remaining gap: a wrong reading of the user's words, such as wrapping the week, is only visible to a reader.

4.7 Reporting: 4/5

- **Before:** reports were assembled by the model from raw JSON and had factual slips: demand missing for 10 of 12 customers, and a broken request put on the wrong day.
- **Now:** `read_result` gives the model a joined, exact account: each goal and its parts, the exact cells behind each goal, every decision with its records' names and numbers, totals per record, and the rules held, tight or broken. The retest reports were correct.

4.8 Speed: 2/5

- 6–12 minutes per problem on the final build; 15–25 minutes before. Almost all of it is Qwen's thinking. Solving takes under a second.
- Mitigations in place: a reply that runs too long is retried with thinking off, earlier specs are compressed, and the repairs avoid extra rounds.

4.9 Robustness: 4/5

Three ways a turn could crash or end without a usable answer were found and fixed:

- **Result save overflow:** the run solved, then crashed while saving the result.
- **Context overflow:** 24,577 + 8,192 tokens exceeded the 32,768 limit. Now prevented by compressing old specs and counting tokens more cautiously, with a retry if the server still refuses.
- **Empty answers after the retry budget ran out:** fixed with a budget per kind of correction, and by taking calls from the model's thinking.

Also fixed: re-queuing a failed run 12 times, empty answers, and a 28,000-character ramble shown to the user.

4.10 Engineering quality: 5/5

- 4,779 backend and 3,379 frontend tests pass. The remaining failures need services absent from the test container (ClickHouse, WeasyPrint, deploy files) or come from a Pydantic version difference in a file not touched here.
- Every fix has a regression test; `test_agent_field_test.py` holds 26 tests that replay the field-test failures.
- Changes are documented (`docs/assistant.md` , `docs/assistant-field-test-2026-10.md`).

5. Defects found and fixed

#	Severity	Area	Defect	Fix
1	Critical	Solver core	Objective $\geq 1e9$ crashed the run after solving	Migration 0107, <code>numeric(24,6)</code> ; real error recorded
2	Critical	Verification	Plan using a never-loaded parameter accepted	Refused, with how to compute it
3	Critical	Verification	Plan loading data no rule reads accepted (wrong goal)	Refused
4	Critical	Verification	Rate for all records / reversed rule passed as "night premium"	Read-back to the model and in the plan
5	High	Robustness	Context overflow ended the turn	Old specs compressed; 2.0 chars/token; retry on refusal
6	High	Robustness	Thinking ran to the limit; 28k characters shown as the answer	Never shown; retried without thinking; honest fallback
7	High	Robustness	Failed run re-queued 12 times	Max 2 per scenario per turn; failure note
8	High	Robustness	Shared correction budget exhausted; empty answer ended the turn	Budget per kind; empty replies sent back
9	High	Data	Mental arithmetic (2,450 for 2,850; 11 for 13; 24 for 23)	Exact totals; <code>query_file</code>
10	High	Reporting	Report missing data / wrong day	<code>read_result</code> with cells behind each goal
11	Medium	Language	No way seen to limit pairs by a value	Pair pattern in the prompt
12	Medium	Language	<code>x[n,d,"night"]</code> refused repeatedly	Bound automatically; message shows how
13	Medium	Language	A list of pairs had no value column	<code>"value": 1</code>
14	Medium	Language	Consecutive days, rate by kind, ranked goals unclear	Prompt patterns
15	Medium	Parsing	Duplicate keys refused 3×	Lists joined, model told
16	Medium	Parsing	Closers out of order in long specs	Repaired (spec tools)
17	Medium	Parsing	<code>"right":</code> key dropped	Repaired (spec tools, native and text calls)
18	Medium	Parsing	Spec written in the reply	Run through <code>check_spec</code>
19	Medium	Parsing	Call left inside the model's thinking	Taken from there
20	Medium	Messages	"mul has exactly two factors"	Shows the nesting
21	Medium	Messages	"no parameter named ..." ×42	Once, with how to declare it
22	Medium	Messages	column "None"	Names the missing field
23	Medium	Messages	"no record ['day','Sun']" ×21	Once, with where records come from
24	Medium	Messages	Terse filter / term-kind / unbound-index refusals	Say what was given and the fix (server and Model Editor)
25	Medium	Messages	Domain name taken: raw Postgres text	Names the domain id and the choices
26	Low	Data	Repeated key in a sheet → duplicate records	One record when fields agree

#	Severity	Area	Defect	Fix
27	Low	Interview	Over-asking (coordinates, names, answered items)	Prompt rule
28	Low	Language	note on a goal term refused	Dropped quietly
29	Low	Language	One goal split into ranked parts	Prompt: one whole goal per rank
30	Low	Reporting	"Tight" reported for equalities; commas inside numbers in tables	Fixed

6. Remaining risks and limits

1. **Semantic modelling errors need a reader.** The read-back makes them visible, but a person must read it before approving. In Retest A, the summary and the model disagreed.
2. **Speed with Qwen 3.5.** 6–12 minutes per problem, and each correction costs a long reply.
3. **Context.** At 32k tokens, long interviews plus specs sit near the limit. The safeguards now prevent crashes, but older detail is dropped.
4. **Tested breadth.** Two problem classes (facility location; scheduling) were tested end to end. Routing, blending/LP, multi-period inventory and GIS-driven site selection have not yet been field-tested this way.
5. **Leftover test data.** The test domains ("Egypt Food Distribution", "Retest Distribution", "Clinic ...", "Retest Clinic ...") are in the database.

7. Recommendations

Priority	Recommendation	Why
1	Make reading As built part of approval in the UI (highlight it; ask "does this match?")	The last line of defence against semantic slips
2	Field-test 3 more classes: an LP blend, a vehicle route, a map-based site selection	Breadth: each class surfaced new failure modes
3	Try <code>LLM_THINKING=off</code> for the Describe tab, or a thinking budget	Most of the time is thinking; measure accuracy against these two references
4	Keep <code>test_agent_field_test.py</code> growing with each new failure	It turns field failures into permanent checks
5	A larger-context or stronger model when available	Fewer rounds, fewer slips, room for longer interviews
6	Store each field test's reference model in a suite	<code>suite_case</code> exists: make these regression cases for the solver too
7	Treat lat/lon CSVs as plain data unless asked to map them	Removes noise columns the model must ignore

Appendix A: Reference models

Warehouses. Decisions: `open[w]` (binary) and `assign[w,c]` (binary). Minimise $\sum \text{fixed_w} \cdot \text{open_w} + \sum \text{cost_wc} \cdot \text{demand_c} \cdot \text{assign_wc}$, subject to:

- $\sum_w \text{assign_wc} = 1$ for every customer;

- $\sum_c \text{demand}_c \cdot \text{assign_wc} \leq \text{cap_w} \cdot \text{open_w};$
- $\sum \text{open} \leq 3;$
- $\text{km_wc} \cdot \text{assign_wc} \leq 450.$

Roster. Decisions: `work[n,d,s]` (binary). Constraints:

- $\sum_n \text{work} \geq \text{needed_ds};$
- $\sum \text{over seniors of work} \geq \text{seniors_ds};$
- $\sum_s \text{work} \leq 1;$
- $\sum_{d,s} \text{work} \leq \text{max_n};$
- $\text{work}[n,d,\text{night}] + \text{work}[n,d+1,\text{morning}] \leq 1.$

Goals in order: (1) $\sum \text{asked_off_nd} \cdot \text{work_nds};$ (2) $\sum \text{wage_n} \cdot \text{factor_s} \cdot \text{work_nds},$ with factor 1.3 for night and 1 otherwise.

Appendix B: Where things are

What	Where
Assistant code	<code>backend/app/agent/</code> (<code>core.py</code> , <code>files.py</code> , <code>toolcall.py</code> , <code>result.py</code> , <code>readback.py</code>)
Wider quantities	<code>backend/alembic/versions/0107_wider_quantities.py</code>
Field-test regression tests	<code>backend/tests/test_agent_field_test.py</code> (26 tests)
Field-test notes	<code>docs/assistant-field-test-2026-10.md</code>
Assistant documentation	<code>docs/assistant.md</code>